

AWS EKS Complete Deployment Guide

1. Create EKS Cluster

Use the following commands to create an EKS cluster:

```
eksctl create cluster \
--name my-eks-cluster \
--region us-east-1 \
--nodegroup-name workers \
--node-type t2.medium \
--nodes 2
```

Cheapest option for development/testing:

```
eksctl create cluster \
--name my-eks-cluster \
--region us-east-1 \
--nodegroup-name app-workers \
--node-type t3.small \
--nodes 1 \
--nodes-min 1 \
--nodes-max 2 \
--managed \
--spot
```

2. Create ECR and Push Images

```
aws ecr create-repository --repository-name backend-app --region us-east-1
aws ecr create-repository --repository-name frontend-app --region us-east-1
```

```
docker tag backend-app:latest <account-id>.dkr.ecr.us-east-1.amazonaws.com/backend-app:latest
docker push <account-id>.dkr.ecr.us-east-1.amazonaws.com/backend-app:latest
```

```
docker tag frontend-app:latest <account-id>.dkr.ecr.us-east-1.amazonaws.com/frontend-app:latest
docker push <account-id>.dkr.ecr.us-east-1.amazonaws.com/frontend-app:latest
```

3. Kubernetes Deployment & Service

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
spec:
  replicas: 1
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend
          image: <your-backend-ecr-image>
```

```

        ports:
        - containerPort: 3000

apiVersion: v1
kind: Service
metadata:
  name: backend-service
spec:
  selector:
    app: backend
  ports:
    - port: 80
      targetPort: 3000
  type: ClusterIP

kubectl apply -f backend-deployment.yaml
kubectl apply -f backend-service.yaml

```

4. Debugging Commands

```

kubectl get nodes
kubectl get pods
kubectl get svc
kubectl logs <pod-name>
kubectl describe pod <pod-name>

```

5. Port Forwarding

```

kubectl port-forward service/backend-service 5000:80
http://localhost:5000

```

6. BusyBox Testing

```

kubectl run test --rm -it --image=busybox -- sh
wget -qO- http://frontend-service
wget -qO- http://backend-service

```

7. Install Ingress (ALB Controller)

```

curl -O https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-controller/main/docs/install/
aws iam create-policy \
  --policy-name AWSLoadBalancerControllerIAMPolicy \
  --policy-document file://iam_policy.json

eksctl utils associate-iam-oidc-provider \
  --region us-east-1 \
  --cluster my-eks-cluster \
  --approve

eksctl create iamserviceaccount \
  --cluster my-eks-cluster \
  --namespace kube-system \
  --name aws-load-balancer-controller \
  --attach-policy-arn arn:aws:iam::<ACCOUNT_ID>:policy/AWSLoadBalancerControllerIAMPolicy \
  --approve

curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash

```

```
helm repo update

helm install aws-load-balancer-controller eks/aws-load-balancer-controller \
-n kube-system \
--set clusterName=my-eks-cluster \
--set serviceAccount.create=false \
--set serviceAccount.name=aws-load-balancer-controller
```

8. Ingress Configuration

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
  annotations:
    alb.ingress.kubernetes.io/scheme: internet-facing
spec:
  ingressClassName: alb
  rules:
    - http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend-service
                port:
                  number: 80

kubectl apply -f ingress.yaml
```

9. Cleanup / Delete Resources

```
kubectl delete deployment frontend
kubectl delete deployment backend
kubectl delete service frontend-service
kubectl delete service backend-service
kubectl delete ingress app-ingress

helm uninstall aws-load-balancer-controller -n kube-system

eksctl delete cluster --name my-eks-cluster --region us-east-1
```